CS6370: Natural Language Processing
Project

Release Date: 26th March 2024 **Deadline:** 14th May 2025

Name:PALI PRAVEEN **Roll No.:CS25M035**

Name:MALEPATI JAYYANTH SAI **Roll No.:CS25M029**

Name:BANTU VIJAYENDRA VARMA **Roll No.:CS25M016**

Name:BUDDA AJAY KUMAR REDDY **Roll No.:CS25M018**

General Instructions:

1. The template for the code (in Python) is provided in a separate zip file. You are expected to fill in the template wherever instructed. Note that any Python library, such as `nltk`, `stanfordcorenlp`, `spacy`, etc., can be used.
2. A folder named `Roll_number.zip` that contains a zip of the code folder and your responses to the questions (a PDF of this document with the solutions written in the text boxes) must be uploaded on Moodle by the deadline.
3. Any submissions made after the deadline will not be graded.
4. Answer the theoretical questions concisely. All the codes should contain proper comments.
5. For questions involving coding components, paste a screenshot of the code.
6. The institute's academic code of conduct will be strictly enforced.

The first assignment in the NLP course involved building a basic text processing module that implements sentence segmentation, tokenization, stemming/lemmatization, stopword removal, and some aspects of spell check. This module involves implementing an Information Retrieval system using the Vector Space Model. The same dataset as in Part 1 (Cranfield dataset) will be used for this purpose.

The project is split into two components — the first is a warm-up component comprising Parts 1 through 4, which acts as a precursor to the second and main component, where you improve over the basic IR system.

Part 1: Working out a Toy IR System (Ambiguity Focus)

Dataset

Consider the following three documents:

- d_1: The star in our solar system provides heat and light.
- d_2: That Hollywood star walked the red carpet for the movie premiere.
- d_3: Astronomers observe distant stars and galaxies using telescopes.

1. Preprocessing and Inverted Index

Assuming the stop words are {*the, in, our, and, that, for*}:

- Perform tokenization.
- Remove stop words.
- Construct the inverted index for the processed documents.

Solution:

Tokenization

After performing tokenization,

Document d1:
[the, star, in, our, solar, system, provides, heat, and, light]

Document d2:
[that, hollywood, star, walked, the, red, carpet, for, the, movie, premiere]

Document d3:
[astronomers, observe, distant, stars, and, galaxies, using, telescopes]

Stopword Removal

Given stopwords: {the, in, our, and, that, for}

After performing stopword removal,

Document d1:
[star, solar, system, provides, heat, light]

Document d2:
[hollywood, star, walked, red, carpet, movie, premiere]

Document d3:

[astronomers, observe, distant, stars, galaxies, using, telescopes]

Inverted Index

Term	Documents
astronomers	d3
carpet	d2
distant	d3
galaxies	d3
heat	d1
hollywood	d2
light	d1
movie	d2
observe	d3
premiere	d2
provides	d1
red	d2
solar	d1
star	d1,d2
stars	d3
system	d1
telescopes	d3
using	d3
walked	d2

2. TF-IDF Term-Document Matrix

- Construct the TF-IDF representation for the corpus $\{d_1, d_2, d_3\}$.
- Clearly show:
  - Term Frequencies (TF)
  - Inverse Document Frequencies (IDF)
  - Final TF-IDF weights

Solution:

After Stopword Removal,

D1 : star solar system provides heat light

D2 : hollywood star walked red carpet movie premiere

D3 : astronomers observe distant stars galaxies using telescopes

TF-IDF Table

Terms	**Counts, tf$i$**			**df$i$**	**D/df$i$**	**IDFi**	**Weights,** $w_i = tfi \times IDF_i$		
	D1	**D2**	**D3**				**D1**	**D2**	**D3**
astronomers	0	0	1	1	$3/1 = 3$	0.4771	0	0	0.4771
carpet	0	1	0	1	$3/1 = 3$	0.4771	0	0.4771	0
distant	0	0	1	1	$3/1 = 3$	0.4771	0	0	0.4771
galaxies	0	0	1	1	$3/1 = 3$	0.4771	0	0	0.4771
heat	1	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0
hollywood	0	1	0	1	$3/1 = 3$	0.4771	0	0.4771	0
light	1	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0
movie	0	1	0	1	$3/1 = 3$	0.4771	0	0.4771	0
observe	0	0	1	1	$3/1 = 3$	0.4771	0	0	0.4771
premiere	0	1	0	1	$3/1 = 3$	0.4771	0	0.4771	0
provides	1	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0
red	0	1	0	1	$3/1 = 3$	0.4771	0	0.4771	0
solar	1	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0
star	1	1	0	2	$3/2 = 1.5$	0.1761	0.1761	0.1761	0
stars	0	0	1	1	$3/1 = 3$	0.4771	0	0	0.4771
system	1	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0
telescopes	0	0	1	1	$3/1 = 3$	0.4771	0	0	0.4771
using	0	0	1	1	$3/1 = 3$	0.4771	0	0	0.4771
walked	0	1	0	1	$3/1 = 3$	0.4771	0	0.4771	0

3. Boolean Retrieval

Consider the query: **"star light"**

- Retrieve documents using the inverted index.
- Which documents match the query terms?

Solution:

Given query:

"star light"

After performing tokeninization and stopword removal, query is

[star, light]

Using the inverted index:

Term	Documents
star	d1, d2
light	d1

Documents Corresponding to term **star** are **d1,d2**

Document Corresponding to term **light** is **d1**

Document **d1** contains both the query terms **star** and **light**

4. Cosine Similarity and Ranking

- Represent the query using TF-IDF.
- Compute cosine similarity between the query and each document.
- Rank the documents based on similarity scores.

Answer the following:

- What is the final ranking?
- Is the ranking desirable? Justify your answer.

Solution:

For the query: **star light**

Term	TF	IDF	TF-IDF
star	1	0.1761	0.1761
light	1	0.4771	0.4771

Now entire tf-idf table is

Terms	**Counts, $\mathbf{tf}_i$**				**$\mathbf{df}_i$**	**$\mathbf{D/df}_i$**	**$\mathbf{IDF}_i$**	**Weights, $w_i = tf_i \times IDF_i$**			
	D1	**D2**	**D3**	**Q**				**D1**	**D2**	**D3**	**Q**
astronomers	0	0	1	0	1	$3/1 = 3$	0.4771	0	0	0.4771	0
carpet	0	1	0	0	1	$3/1 = 3$	0.4771	0	0.4771	0	0
distant	0	0	1	0	1	$3/1 = 3$	0.4771	0	0	0.4771	0
galaxies	0	0	1	0	1	$3/1 = 3$	0.4771	0	0	0.4771	0
heat	1	0	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0	0
hollywood	0	1	0	0	1	$3/1 = 3$	0.4771	0	0.4771	0	0
light	1	0	0	1	1	$3/1 = 3$	0.4771	0.4771	0	0	0.4771
movie	0	1	0	0	1	$3/1 = 3$	0.4771	0	0.4771	0	0
observe	0	0	1	0	1	$3/1 = 3$	0.4771	0	0	0.4771	0
premiere	0	1	0	0	1	$3/1 = 3$	0.4771	0	0.4771	0	0
provides	1	0	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0	0
red	0	1	0	0	1	$3/1 = 3$	0.4771	0	0.4771	0	0
solar	1	0	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0	0
star	1	1	0	1	2	$3/2 = 1.5$	0.1761	0.1761	0.1761	0	0.1761
stars	0	0	1	0	1	$3/1 = 3$	0.4771	0	0	0.4771	0
system	1	0	0	0	1	$3/1 = 3$	0.4771	0.4771	0	0	0
telescopes	0	0	1	0	1	$3/1 = 3$	0.4771	0	0	0.4771	0
using	0	0	1	0	1	$3/1 = 3$	0.4771	0	0	0.4771	0
walked	0	1	0	0	1	$3/1 = 3$	0.4771	0	0.4771	0	0

$$D_1 = (0, 0, 0, 0, 0.4771, 0, 0.4771, 0, 0, 0, 0.4771, 0, 0.4771, 0.1761, 0, 0.4771, 0, 0, 0)$$

$$D_2 = (0, 0.4771, 0, 0, 0, 0.4771, 0, 0.4771, 0, 0.4771, 0, 0.4771, 0, 0.1761, 0, 0, 0, 0, 0.4771)$$

$$D_3 = (0.4771, 0, 0.4771, 0.4771, 0, 0, 0, 0, 0.4771, 0, 0, 0, 0, 0, 0.4771, 0, 0.4771, 0.4771, 0)$$

$$Q = (0, 0, 0, 0, 0, 0, 0.4771, 0, 0, 0, 0, 0, 0, 0.1761, 0, 0, 0, 0, 0)$$

Cosine Similarity

Cosine similarity is calculated using:

$$\cos(\theta) = \frac{D \cdot Q}{|D||Q|}$$

$$|Q| = \sqrt{0.1761^2 + 0.4771^2}$$

$$|Q| = \sqrt{0.0310 + 0.2276}$$

$$|Q| = \sqrt{0.2586}$$

$$|Q| \approx 0.5085$$

Similarity with d1

Dot product of D1 and Q:

$$D_1 \cdot Q = (0.1761)(0.1761) + (0.4771)(0.4771)$$

$$= 0.0310 + 0.2276$$

$$= 0.2586$$

$$|D_1| = \sqrt{0.1761^2 + 5(0.4771^2)}$$

$$= \sqrt{0.0310 + 1.1380}$$

$$= \sqrt{1.1690}$$

$$|D_1| \approx 1.0812$$

Cosine similarity:

$$\cos(D_1, Q) = \frac{0.2586}{(1.0812)(0.5085)}$$

$$\cos(D_1, Q) \approx 0.470$$

Similarity with d2

Dot product of D2 and Q:

$$D_2 \cdot Q = (0.1761)(0.1761)$$

$$= 0.0310$$

$$|D_2| = \sqrt{0.1761^2 + 6(0.4771^2)}$$

$$= \sqrt{0.0310 + 1.3656}$$

$$= \sqrt{1.3966}$$

$$|D_2| \approx 1.1818$$

Cosine similarity:

$$\cos(D_2, Q) = \frac{0.0310}{(1.1818)(0.5085)}$$

$$\cos(D_2, Q) \approx 0.052$$

Similarity with d3

No query term appears in d3.

$$\cos(D_3, Q) = 0$$

Final Ranking

Rank	Document	Similarity Score
1	d1	0.470
2	d2	0.052
3	d3	0

Is the Ranking Desirable?

Yes, the ranking is desirable.

- d1 is ranked highest because it contains both query terms "star" and "light".
- d2 contains only the term "star", so it receives a lower similarity score.
- d3 does not contain any query term and therefore receives zero similarity.

The ranking correctly identifies d1 as the most relevant document for the query.

5. Analysis of Word Sense Ambiguity

Consider the query: **"movie star"**

- Which document(s) should ideally be retrieved?
- Does the system retrieve the correct document(s)?
- Explain how word sense ambiguity ("star") affects retrieval.

Solution:
Given Query,

"movie star"

After tokenization and stopword removal query becomes,

[movie, star]

Which document(s) should ideally be retrieved?

The document that should ideally be retrieved is **d2** because d2 discusses a Hollywood celebrity and contains both the terms **movie** and **star**

Does the system retrieve the correct document(s)?

Yes, the system retrieves d2 because it contains both query terms "movie" and "star".

However, the system may also retrieve d1 because d1 contains the word "star".

Thus, although d2 is the most relevant document, d1 may also appear in the retrieval results due to the shared word "star".

Effect of Word Sense Ambiguity

The word "star" has multiple meanings:

- Astronomical object (d1)
- Celebrity or actor (d2)

This creates **word sense ambiguity**.

The Vector Space Model treats words purely as tokens and does not understand semantic meaning or context. Therefore, the system considers both occurrences of "star" as identical terms even though their meanings are different.

As a result,

- Relevant documents may be mixed with partially irrelevant documents.
- Retrieval accuracy decreases because the system cannot distinguish between different senses of the same word.

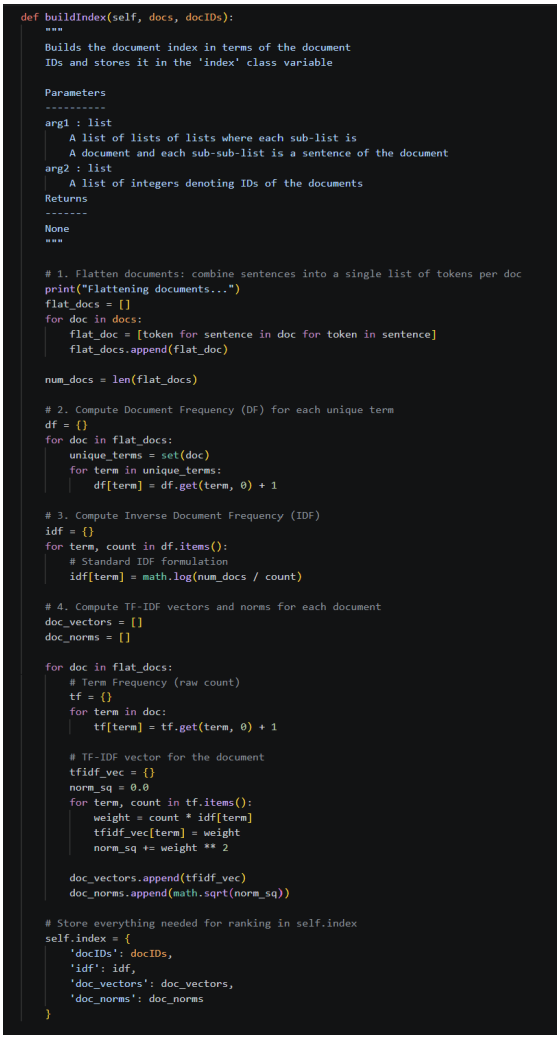
In this example, the query "movie star" is intended to refer to a celebrity, but the system may still retrieve d1 because it also contains the term "star".

Part 2: Building and Extending an IR System [Implementation]

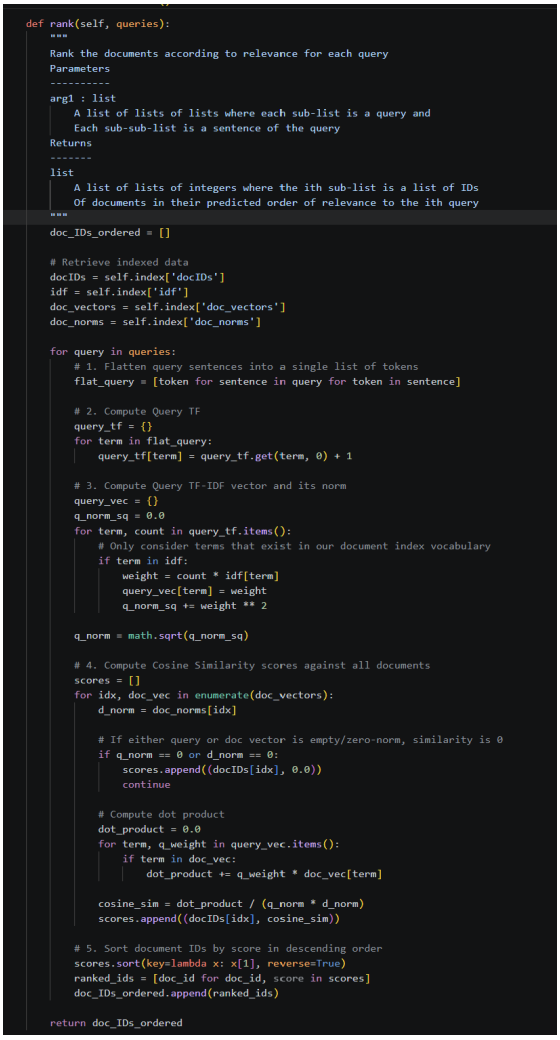
1. Implement the retrieval component of the Information Retrieval (IR) system using the template provided. Represent documents using the TF-IDF vector space model.

Solution: The Information Retrieval (IR) system has been implemented using the Vector Space Model with TF–IDF weighting. The system performs the following steps:

1. **Index Building:** For each preprocessed document, raw term frequencies are computed, IDF values are computed using $\text{idf}(t) = \log(N/\text{df}(t))$, and a TF–IDF vector is stored along with its L2 norm for fast cosine computation.
2. **Query Ranking:** Each preprocessed query is projected into the same TF–IDF vector space (terms not in the document vocabulary are silently dropped). Cosine similarity is computed against every document, and documents are returned in descending order of similarity.



```
def buildIndex(self, docs, docIDs):
    """
    Builds the document index in terms of the document
    IDs and stores it in the 'index' class variable

    Parameters
    ----------
    arg1 : list
        A list of lists of lists where each sub-list is
        A document and each sub-sub-list is a sentence of the document
    arg2 : list
        A list of integers denoting IDs of the documents
    Returns
    -------
    None
    """

    # 1. Flatten documents: combine sentences into a single list of tokens per doc
    print("Flattening documents...")
    flat_docs = []
    for doc in docs:
        flat_doc = [token for sentence in doc for token in sentence]
        flat_docs.append(flat_doc)

    num_docs = len(flat_docs)

    # 2. Compute Document Frequency (DF) for each unique term
    df = {}
    for doc in flat_docs:
        unique_terms = set(doc)
        for term in unique_terms:
            df[term] = df.get(term, 0) + 1

    # 3. Compute Inverse Document Frequency (IDF)
    idf = {}
    for term, count in df.items():
        # Standard IDF formulation
        idf[term] = math.log(num_docs / count)

    # 4. Compute TF-IDF vectors and norms for each document
    doc_vectors = []
    doc_norms = []

    for doc in flat_docs:
        # Term Frequency (raw count)
        tf = {}
        for term in doc:
            tf[term] = tf.get(term, 0) + 1

        # TF-IDF vector for the document
        tfidf_vec = {}
        norm_sq = 0.0
        for term, count in tf.items():
            weight = count * idf[term]
            tfidf_vec[term] = weight
            norm_sq += weight ** 2

        doc_vectors.append(tfidf_vec)
        doc_norms.append(math.sqrt(norm_sq))

    # Store everything needed for ranking in self.index
    self.index = {
        'docIDs': docIDs,
        'idf': idf,
        'doc_vectors': doc_vectors,
        'doc_norms': doc_norms
    }
```



```
def rank(self, queries):
    """
    Rank the documents according to relevance for each query
    Parameters
    ----------
    arg1 : list
        A list of lists of lists where each sub-list is a query and
        Each sub-sub-list is a sentence of the query
    Returns
    -------
    list
        A list of lists of integers where the ith sub-list is a list of IDs
        Of documents in their predicted order of relevance to the ith query
    """
    doc_IDs_ordered = []

    # Retrieve indexed data
    docIDs = self.index['docIDs']
    idf = self.index['idf']
    doc_vectors = self.index['doc_vectors']
    doc_norms = self.index['doc_norms']

    for query in queries:
        # 1. Flatten query sentences into a single list of tokens
        flat_query = [token for sentence in query for token in sentence]

        # 2. Compute Query TF
        query_tf = {}
        for term in flat_query:
            query_tf[term] = query_tf.get(term, 0) + 1

        # 3. Compute Query TF-IDF vector and its norm
        query_vec = {}
        q_norm_sq = 0.0
        for term, count in query_tf.items():
            # Only consider terms that exist in our document index vocabulary
            if term in idf:
                weight = count * idf[term]
                query_vec[term] = weight
                q_norm_sq += weight ** 2

        q_norm = math.sqrt(q_norm_sq)

        # 4. Compute Cosine Similarity scores against all documents
        scores = []
        for idx, doc_vec in enumerate(doc_vectors):
            d_norm = doc_norms[idx]

            # If either query or doc vector is empty/zero-norm, similarity is 0
            if q_norm == 0 or d_norm == 0:
                scores.append((docIDs[idx], 0.0))
                continue

            # Compute dot product
            dot_product = 0.0
            for term, q_weight in query_vec.items():
                if term in doc_vec:
                    dot_product += q_weight * doc_vec[term]

            cosine_sim = dot_product / (q_norm * d_norm)
            scores.append((docIDs[idx], cosine_sim))

        # 5. Sort document IDs by score in descending order
        scores.sort(key=lambda x: x[1], reverse=True)
        ranked_ids = [doc_id for doc_id, score in scores]
        doc_IDs_ordered.append(ranked_ids)

    return doc_IDs_ordered
```

Part 3: Evaluating your IR System [Implementation]

1. Implement the following evaluation measures in the template provided:
    - Precision@k
    - Recall@k
    - $F_{0.5}$-score@k
    - Average Precision (AP@k)
    - nDCG@k

   Report the following:
    - Precision@k:
    - Recall@k:
    - $F_{0.5}$-score@k:
    - AP@k:
    - nDCG@k:

Solution:

Formulas

- **Precision@k**: $\mathrm{P@}k = \frac{|\{\text{relevant docs}\} \cap \{\text{top-}k \text{ retrieved}\}|}{k}$
- **Recall@k**: $\mathrm{R@}k = \frac{|\{\text{relevant docs}\} \cap \{\text{top-}k \text{ retrieved}\}|}{|\{\text{relevant docs}\}|}$
- **$\mathbf{F_{0.5}}$-score@k** (weighted harmonic mean with $\beta = 0.5$):

$$F_{0.5}@k = (1 + \beta^2) \cdot \frac{P \cdot R}{\beta^2 P + R} = 1.25 \cdot \frac{P \cdot R}{0.25P + R}$$

- **Average Precision (AP@k)**: average of P@i over the ranks $i \leq k$ at which a relevant document is retrieved, divided by the total number of relevant documents.
- **nDCG@k**: DCG@k/IDCG@k, where $\mathrm{DCG@}k = \sum_{i=1}^{k} \mathrm{gain}(i)/\log_2(i+1)$. Cranfield relevance positions (1 to 4) are mapped to gains $5 - p$, so position 1 (most relevant) gives gain 4.

The complete code is shown below as a screenshot:

```python
def queryPrecision(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):
    """
    Computation of precision of the Information Retrieval System
    at a given value of k for a single query

    Parameters
    ----------
    arg1 : list
        A list of integers denoting the IDs of documents in
        their predicted order of relevance to a query
    arg2 : int
        The ID of the query in question
    arg3 : list
        The list of IDs of documents relevant to the query (ground truth)
    arg4 : int
        The k value

    Returns
    -------
    float
        The precision value as a number between 0 and 1
    """
    if k <= 0:
        return 0.0

    # 1. Get the top k retrieved document IDs
    top_k_retrieved = query_doc_IDs_ordered[:k]

    if not top_k_retrieved:
        return 0.0

    # 2. Convert ground truth list to a set for O(1) lookup time
    true_docs_set = set(true_doc_IDs)

    # 3. Count how many of the top k retrieved documents are relevant
    relevant_count = sum(1 for doc_id in top_k_retrieved if doc_id in true_docs_set)

    # 4. Calculate precision
    precision = relevant_count / k

    return float(precision)
```

```python
def meanPrecision(self, doc_IDs_ordered, query_ids, qrels, k):
    """
    Computation of precision of the Information Retrieval System
    at a given value of k, averaged over all the queries
    """
    meanPrecision = -1

    # 1. Edge case handling
    if not query_ids or k <= 0:
        return 0.0

    # 2. Map ground-truth relevant documents for each query.
    # Grouping by query_num allows O(1) retrieval during the loop.
    true_docs_map = {}
    for qrel in qrels:
        # Extract query ID and relevant document ID from the dictionary
        q_id = int(qrel['query_num'])
        doc_id = int(qrel['id'])

        if q_id not in true_docs_map:
            true_docs_map[q_id] = []
        true_docs_map[q_id].append(doc_id)

    # 3. Iterate over all provided query IDs and calculate their individual P@k
    total_precision = 0.0

    for idx, q_id in enumerate(query_ids):
        q_id_int = int(q_id)

        # Retrieve predicted ordering for this specific query
        predicted_docs = doc_IDs_ordered[idx]
        predicted_docs = [int(doc) for doc in predicted_docs]

        # Retrieve ground truth relevant docs (default to empty list if none exist)
        true_docs = true_docs_map.get(q_id_int, [])

        # Utilize the queryPrecision function to evaluate this single query
        q_prec = self.queryPrecision(predicted_docs, q_id_int, true_docs, k)
        total_precision += q_prec

    # 4. Compute the average precision across the total number of queries
    meanPrecision = total_precision / len(query_ids)

    return float(meanPrecision)
```

```python
def queryRecall(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):
    """
    Computation of recall of the Information Retrieval System
    at a given value of k for a single query
    """
    # Edge case: if there are no true relevant documents or k is invalid
    if not true_doc_IDs or k <= 0:
        return 0.0

    # 1. Get the top k retrieved document IDs
    top_k_retrieved = query_doc_IDs_ordered[:k]

    # 2. Convert ground truth list to a set for O(1) lookup time
    true_docs_set = set(true_doc_IDs)

    # 3. Count how many of the top k retrieved documents are relevant
    relevant_count = sum(1 for doc_id in top_k_retrieved if doc_id in true_docs_set)

    # 4. Calculate recall: fraction of total relevant documents successfully retrieved
    recall = relevant_count / len(true_doc_IDs)

    return float(recall)
```

```python
def meanRecall(self, doc_IDs_ordered, query_ids, qrels, k):
    """
    Computation of recall of the Information Retrieval System
    at a given value of k, averaged over all the queries
    """
    # 1. Edge case handling
    if not query_ids or k <= 0:
        return 0.0

    # 2. Map ground-truth relevant documents for each query using qrels.
    # Grouping by query_num allows O(1) retrieval during the loop.
    true_docs_map = {}
    for qrel in qrels:
        q_id = int(qrel['query_num'])
        doc_id = int(qrel['id'])

        if q_id not in true_docs_map:
            true_docs_map[q_id] = []
        true_docs_map[q_id].append(doc_id)

    # 3. Iterate over all provided query IDs and calculate their individual Recall@k
    total_recall = 0.0

    for idx, q_id in enumerate(query_ids):
        q_id_int = int(q_id)

        # Retrieve predicted ordering for this specific query
        predicted_docs = doc_IDs_ordered[idx]
        predicted_docs = [int(doc) for doc in predicted_docs]

        # Retrieve ground truth relevant docs (default to empty list if none exist)
        true_docs = true_docs_map.get(q_id_int, [])

        # Utilize the queryRecall function to evaluate this single query
        q_rec = self.queryRecall(predicted_docs, q_id_int, true_docs, k)
        total_recall += q_rec

    # 4. Compute the average recall across the total number of queries
    meanRecall = total_recall / len(query_ids)

    return float(meanRecall)
```

```python
def queryFscore(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):
    """
    Computation of F0.5-score of the Information Retrieval System
    at a given value of k for a single query.

    F0.5 uses beta = 0.5, which weights precision twice as heavily as recall.
    Formula: F_beta = (1 + beta^2) * P * R / (beta^2 * P + R)
    """
    # 1. Compute Precision@k and Recall@k using your internal class methods
    precision = self.queryPrecision(query_doc_IDs_ordered, query_id, true_doc_IDs, k)
    recall = self.queryRecall(query_doc_IDs_ordered, query_id, true_doc_IDs, k)

    # 2. Edge case: If both precision and recall are 0, F-score is defined as 0.0
    if precision + recall == 0.0:
        return 0.0

    # 3. Compute F0.5-score (beta = 0.5)
    beta = 0.5
    beta_sq = beta * beta  # 0.25

    # Edge case: avoid division by zero when the denominator collapses
    denominator = beta_sq * precision + recall
    if denominator == 0.0:
        return 0.0

    fscore = (1 + beta_sq) * precision * recall / denominator

    return float(fscore)
```

```python
def meanFscore(self, doc_IDs_ordered, query_ids, qrels, k):
    """
    Computation of F0.5-score of the Information Retrieval System
    at a given value of k, averaged over all the queries.
    """
    # 1. Edge case handling
    if not query_ids or k <= 0:
        return 0.0

    # 2. Map ground-truth relevant documents for each query using qrels.
    # Grouping by query_num allows O(1) retrieval during the loop.
    true_docs_map = {}
    for qrel in qrels:
        q_id = int(qrel['query_num'])
        doc_id = int(qrel['id'])

        if q_id not in true_docs_map:
            true_docs_map[q_id] = []
        true_docs_map[q_id].append(doc_id)

    # 3. Iterate over all provided query IDs and calculate their individual F0.5-score@k
    total_fscore = 0.0

    for idx, q_id in enumerate(query_ids):
        q_id_int = int(q_id)

        # Retrieve predicted ordering for this specific query
        predicted_docs = doc_IDs_ordered[idx]
        predicted_docs = [int(doc) for doc in predicted_docs]

        # Retrieve ground truth relevant docs (default to empty list if none exist)
        true_docs = true_docs_map.get(q_id_int, [])

        # Utilize the queryFscore function to evaluate this single query
        q_f = self.queryFscore(predicted_docs, q_id_int, true_docs, k)
        total_fscore += q_f

    # 4. Compute the average F0.5-score across the total number of queries
    meanFscore = total_fscore / len(query_ids)

    return float(meanFscore)
```

```python
def queryNDCG(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):
    """
    Computation of nDCG of the Information Retrieval System
    at given value of k for a single query
    """
    nDCG = -1

    # 1. Edge case handling
    if k <= 0 or not true_doc_IDs:
        return 0.0

    top_k_retrieved = query_doc_IDs_ordered[:k]
    if not top_k_retrieved:
        return 0.0

    # 2. Calculate Discounted Cumulative Gain (DCG@k)
    dcg = 0.0
    for rank_idx, doc_id in enumerate(top_k_retrieved):
        # true_doc_IDs is our dictionary mapping doc -> gain
        gain = true_doc_IDs.get(doc_id, 0.0)
        if gain > 0:
            dcg += gain / math.log2(rank_idx + 2)

    # 3. Calculate Ideal DCG (IDCG@k)
    # Sort all available ground-truth gains descending
    ideal_gains = sorted(true_doc_IDs.values(), reverse=True)

    idcg = 0.0
    for rank_idx, gain in enumerate(ideal_gains[:k]):
        idcg += gain / math.log2(rank_idx + 2)

    # 4. Normalize
    if idcg == 0.0:
        return 0.0

    nDCG = dcg / idcg
    return float(nDCG)
```

```python
def meanNDCG(self, doc_IDs_ordered, query_ids, qrels, k):
    """
    Computation of nDCG of the Information Retrieval System
    at a given value of k, averaged over all the queries
    """
    meanNDCG = -1

    if not query_ids or k <= 0:
        return 0.0

    # 1. Map each query to a dictionary of {doc_id: relevance_gain}
    true_docs_map = {}
    for qrel in qrels:
        q_id = int(qrel['query_num'])
        doc_id = int(qrel['id'])

        # Invert Cranfield position (1-4) into an IR gain score (4-1)
        pos = int(qrel['position'])
        gain = 5.0 - pos

        if q_id not in true_docs_map:
            true_docs_map[q_id] = {}

        true_docs_map[q_id][doc_id] = gain

    # 2. Accumulate nDCG scores across all queries
    total_ndcg = 0.0
    for idx, q_id in enumerate(query_ids):
        q_id_int = int(q_id)
        predicted_docs = [int(doc) for doc in doc_IDs_ordered[idx]]

        # Extract the dictionary of true gains for this query
        query_true_gains = true_docs_map.get(q_id_int, {})

        # Pass the dictionary cleanly into the 'true_doc_IDs' parameter
        total_ndcg += self.queryNDCG(predicted_docs, q_id_int, query_true_gains, k)

    # 3. Compute the mean
    meanNDCG = total_ndcg / len(query_ids)
    return float(meanNDCG)
```

```python
def queryAveragePrecision(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):
    """
    Computation of average precision of the Information Retrieval System
    at a given value of k for a single query (the average of precision@i
    values for i such that the ith document is truly relevant)
    """
    # 1. Edge case handling: invalid k or no ground-truth relevant documents
    if k <= 0 or not true_doc_IDs:
        return 0.0

    top_k_retrieved = query_doc_IDs_ordered[:k]
    if not top_k_retrieved:
        return 0.0

    # 2. Convert ground truth to a set for O(1) lookup efficiency.
    # This works perfectly whether true_doc_IDs is passed as a list or a dict.
    true_docs_set = set(true_doc_IDs)

    sum_precisions = 0.0
    relevant_hits = 0

    # 3. Single-pass loop to calculate Precision@i at each relevant rank
    for rank_idx, doc_id in enumerate(top_k_retrieved):
        if doc_id in true_docs_set:
            relevant_hits += 1
            # Calculate P@i where i is the 1-based rank (rank_idx + 1)
            precision_at_i = relevant_hits / (rank_idx + 1)
            sum_precisions += precision_at_i

    # 4. Compute Average Precision
    # Standard IR divides by the total number of relevant documents available.
    avgPrecision = sum_precisions / len(true_doc_IDs)

    return float(avgPrecision)
```

```python
def meanAveragePrecision(self, doc_IDs_ordered, query_ids, qrels, k):
    """
    Computation of MAP of the Information Retrieval System
    at given value of k, averaged over all the queries
    """
    map_score = -1

    # 1. Edge case handling
    if not query_ids or k <= 0:
        return 0.0

    # 2. Map ground-truth relevant documents for each query using qrels.
    # Grouping by query_num allows O(1) retrieval during the loop.
    true_docs_map = {}
    for qrel in qrels:
        q_id = int(qrel['query_num'])
        doc_id = int(qrel['id'])

        if q_id not in true_docs_map:
            true_docs_map[q_id] = []
        true_docs_map[q_id].append(doc_id)

    # 3. Iterate over all provided query IDs and calculate their individual AP@k
    total_ap = 0.0

    for idx, q_id in enumerate(query_ids):
        q_id_int = int(q_id)

        # Retrieve predicted ordering for this specific query safely cast to ints
        predicted_docs = [int(doc) for doc in doc_IDs_ordered[idx]]

        # Retrieve ground truth relevant docs (default to empty list if none exist)
        true_docs = true_docs_map.get(q_id_int, [])

        # Utilize the queryAveragePrecision function to evaluate this single query
        q_ap = self.queryAveragePrecision(predicted_docs, q_id_int, true_docs, k)
        total_ap += q_ap

    # 4. Compute the Mean Average Precision across all queries
    map_score = total_ap / len(query_ids)

    return float(map_score)
```

```python
def queryReciprocalRank(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):
    """
    Computation of reciprocal rank for a single query

    Parameters
    ----------
    arg1 : list
        Ranked list of document IDs
    arg2 : int
        Query ID
    arg3 : list
        List of relevant document IDs
    arg4 : int
        The k value

    Returns
    -------
    float
        Reciprocal rank value
    """
    # 1. Edge case handling: invalid k or missing data
    if k <= 0 or not true_doc_IDs or not query_doc_IDs_ordered:
        return 0.0

    # 2. Extract the top k retrieved document IDs
    top_k_retrieved = query_doc_IDs_ordered[:k]

    # 3. Convert ground truth to a set for O(1) lookup efficiency.
    # This works perfectly whether true_doc_IDs is passed as a list or a dict keys view.
    true_docs_set = set(true_doc_IDs)

    # 4. Find the first relevant document
    for rank_idx, doc_id in enumerate(top_k_retrieved):
        if doc_id in true_docs_set:
            # The 1-based rank is rank_idx + 1
            reciprocalRank = 1.0 / (rank_idx + 1)
            return float(reciprocalRank)

    # 5. If no relevant document is found within top k results
    return 0.0
```

```python
def meanReciprocalRank(self, doc_IDs_ordered, query_ids, qrels, k):
    """
    Computation of Mean Reciprocal Rank (MRR)
    averaged over all queries

    Parameters
    ----------
    arg1 : list
        List of ranked document lists
    arg2 : list
        Query IDs
    arg3 : list
        Relevance judgments
    arg4 : int
        The k value

    Returns
    -------
    float
        MRR value
    """
    if not query_ids or k <= 0:
        return 0.0

    # 1. Map ground-truth relevant documents for each query using qrels
    true_docs_map = {}
    for qrel in qrels:
        q_id = int(qrel['query_num'])
        doc_id = int(qrel['id'])

        if q_id not in true_docs_map:
            true_docs_map[q_id] = []
        true_docs_map[q_id].append(doc_id)

    # 2. Accumulate Reciprocal Rank scores across all queries
    total_rr = 0.0
    for idx, q_id in enumerate(query_ids):
        q_id_int = int(q_id)
        predicted_docs = [int(doc) for doc in doc_IDs_ordered[idx]]
        true_docs = true_docs_map.get(q_id_int, [])

        total_rr += self.queryReciprocalRank(predicted_docs, q_id_int, true_docs, k)

    # 3. Compute the Mean Reciprocal Rank
    mean_rr = total_rr / len(query_ids)
    return float(mean_rr)
```

2. Report the following global evaluation metrics:
   - **Mean Average Precision (MAP):** Compute MAP by taking the mean of the Average Precision (AP) scores across all queries, where applicable.
   - **Mean Reciprocal Rank (MRR):** Compute MRR by averaging the reciprocal of the rank of the first relevant document for each query, where applicable.

Solution: The global evaluation metrics, computed at $k = 10$ over all 225 Cranfield queries, are:

Metric	Value
Mean Average Precision (MAP)	0.2970
Mean Reciprocal Rank (MRR)	0.7227

Interpretation:

- A MAP of 0.297 means that, on average, the system places approximately 30% of the relevant documents in the top-10 ranked positions, weighted by how high in the ranking each relevant document appears.
- An MRR of 0.723 means that, on average, the rank of the *first* relevant document for each query is approximately $1/0.723 \approx 1.38$. In other words, the system typically returns a relevant document at rank 1 or 2 for most queries, indicating strong top-rank precision.

3. Assume that for a given query, the set of relevant documents is as listed in `incran_qrels.json`. Any document with a relevance score of 1 to 4 is considered as relevant.

   - For each query in the Cranfield dataset, compute Precision, Recall, F-score, Average Precision, and nDCG for $k = 1$ to 10.
   - Average each measure over all queries.
   - Plot each evaluation metric as a function of k using the provided template code.

   Report:

   - Graphs of all evaluation metrics vs k
   - Observations based on the plots

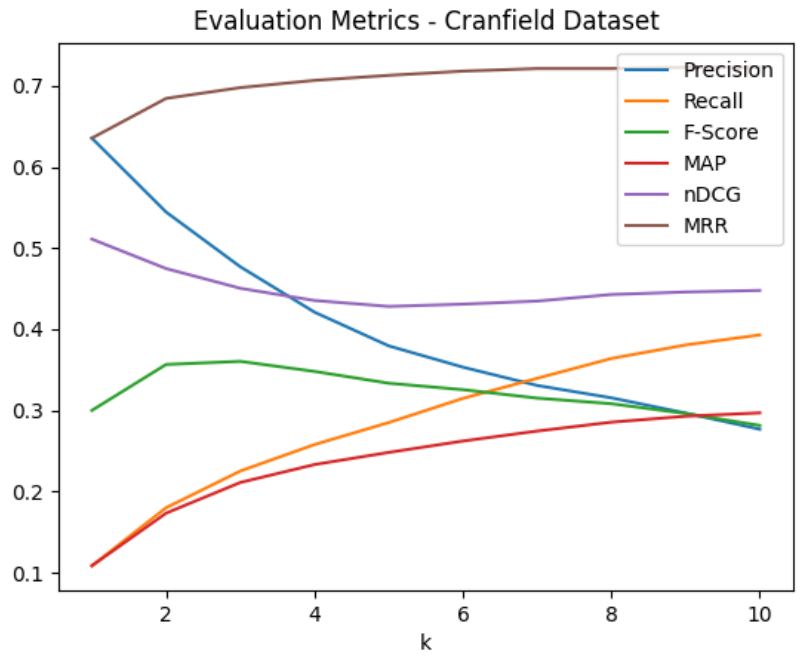
Solution:

Evaluation Metrics for $k = 1$ to 10

The following table reports each metric averaged over all 225 Cranfield queries, evaluated at $k = 1, 2, \ldots, 10$:

k	**Precision**	**Recall**	**$F_{0.5}$-score**	**MAP**	**nDCG**	**MRR**
1	0.6356	0.1086	0.3001	0.1086	0.5111	0.6356
2	0.5444	0.1801	0.3567	0.1735	0.4748	0.6844
3	0.4770	0.2254	0.3605	0.2112	0.4505	0.6978
4	0.4211	0.2580	0.3481	0.2333	0.4355	0.7067
5	0.3796	0.2851	0.3335	0.2483	0.4282	0.7129
6	0.3533	0.3149	0.3256	0.2623	0.4310	0.7181
7	0.3308	0.3396	0.3152	0.2746	0.4347	0.7212
8	0.3156	0.3640	0.3083	0.2856	0.4428	0.7212
9	0.2968	0.3807	0.2963	0.2929	0.4459	0.7227
10	0.2769	0.3931	0.2815	0.2970	0.4479	0.7227

Plot of Evaluation Metrics vs k



Observations from the Plots

(a) Precision decreases as k increases. It starts at 0.6356 for k = 1 and drops to 0.2769 at k = 10. This means the top results are usually more relevant, but adding more results includes more irrelevant documents.

(b) Recall increases as k increases, from 0.1086 at k = 1 to 0.3931 at k = 10. This means retrieving more documents helps find more relevant documents.

(c) F0.5-score is highest around k = 2–3 at about 0.36 and then decreases. Since F0.5 gives more importance to precision, the score is best when precision is still high and recall starts improving. This shows the system performs best in the top-ranked results.

(d) MAP increases as k increases, from 0.1086 at k = 1 to 0.2970 at k = 10. This happens because checking more ranks helps find more relevant documents, which improves the score.

(e) nDCG starts high at 0.5111 for k = 1, decreases to about 0.43 around k = 5, and then slightly increases to 0.4479 at k = 10. This shows that the system often retrieves a highly relevant document at the top rank, but as more ranks are considered, the score drops before improving again with additional relevant documents.

(f) MRR increases quickly and reaches about 0.72 by k = 7. This means the first relevant document usually appears within the top one or two results.

(g) Precision and recall become almost equal around k = 7. This means that at this cutoff, the system retrieves nearly the same amount of relevant and irrelevant documents in the top-k results..

Overall, the baseline TF–IDF VSM shows the normal behaviour of a bag-of-words retrieval system. Precision decreases while recall increases as k grows. The high MRR shows that relevant documents are usually found in the top results, but the lower MAP and recall values show that the system misses many other relevant documents. These limitations are what the improvements in Part 5 aim to solve.

4. Using the `time` module in Python, report the run time of your IR system.

Solution:

Stage	**Time (seconds)**
Query preprocessing (225 queries)	2.18
Document preprocessing (1400 docs)	0.97
Index build (TF–IDF + norms)	0.06
Ranking (cosine similarity for all 225 queries)	0.48
Total IR system runtime	**3.69**

The complete IR pipeline runs in under four seconds for 225 queries and 1400 documents, showing that TF–IDF VSM is fast and efficient as a baseline system. Query preprocessing takes the most time (2.18 s, about 59 percent of total runtime) because spaCy and WordNet models are called for every query. Document preprocessing is faster (0.97 s) since the models are already loaded. Index building (0.06 s) and ranking (0.48 s, about 2 ms per query) take very little time.

Part 4: Analysis [Theory]

1. What are the limitations of the Vector Space Model (VSM)? Provide examples from the Cranfield dataset that illustrate these shortcomings in your IR system.
   Your answer should include:
   - Identification of key limitations of VSM
   - Concrete examples from your retrieval results

Solution:

Limitations of the Vector Space Model (VSM)

The Vector Space Model represents documents and queries using TF–IDF weighted bag-of-words vectors and ranks documents using cosine similarity. Although simple and effective, it has several structural limitations

(1) High computation cost – Calculating TF–IDF weights and cosine similarity for large datasets takes significant time and processing power.

(2) Word order is ignored – The model treats documents as bag-of-words, so different sentences with the same words can produce the same result.

(3) No semantic understanding – TF–IDF cannot understand meaning or context, only exact word matches.

(4) Synonymy problem – Relevant documents may use different words with the same meaning (e.g., car and automobile), causing the system to miss them and lowering recall.

(5) Polysemy problem – A word can have multiple meanings, which may retrieve irrelevant documents and reduce precision.

(6) Vocabulary mismatch and preprocessing errors – Different writing styles, stemming, or parsing mistakes can lead to missing relevant documents or retrieving incorrect ones.

Concrete Examples from the Cranfield Retrieval Results

- Synonymy problem (loss of recall): Queries using words like airfoil may miss relevant documents that use aerofoil. Similarly, documents about viscous flow detachment may not be retrieved for queries on laminar boundary layer separation, even though both mean the same thing.
- Polysemy problem (loss of precision): The word flow can mean fluid flow, heat flow, or mass flow. Because the model cannot understand the correct meaning, queries on heat flow may also retrieve unrelated aerodynamic flow documents.
- Word-order insensitivity: Queries with the same words but different order produce identical rankings because the model ignores word order and only considers word frequency.

These are fundamental limitations of VSM and motivate improvements such as LSA, BM25, and hybrid models

2. **The Zero-Result Problem (OOV Issue):**
    - What happens when a query contains a word that does not appear in the document vocabulary (Out-of-Vocabulary term)?
    - How does this affect retrieval results in your system?
    - Provide an example query demonstrating this issue.

Solution:

If a query contains a term that does not appear in the corpus, its document frequency becomes zero, making the IDF value undefined. In our implementation, such terms are ignored and removed from the query vector, so they do not affect the similarity score.

Effect on retrieval results.

- If all query terms are OOV (out-of-vocabulary), every document gets a similarity score of zero, leading to empty or random rankings.
- If only some query terms are OOV, the system ignores them and ranks documents using only the remaining terms, which may miss the user's actual intent.
- Common causes of OOV terms include spelling mistakes, rare technical words, missing named entities, and word forms not handled correctly by stemming.

Example query. For example, in the query "ramjet engine combustor efficiency", if the word ramjet is not present in the corpus, it is ignored. The ranking is then based only on engine, combustor, and efficiency, which may return general engine-related documents instead of ramjet-specific ones. Similarly, a misspelled query like "aerodynammics of slender bodies" silently ignores the incorrect word and reduces retrieval accuracy.

Part 5: Improving the IR System [Project]

Based on the factual record of actual retrieval failures observed in your assignment, develop hypotheses that could address these failures. You may need to identify the implicit assumptions made by your approach that resulted in undesirable outcomes.

To realize improvements, you may use any suitable method(s), including hybrid approaches that combine knowledge from linguistic, background, and introspective sources to represent documents. Some example methods discussed in class include:

- Latent Semantic Analysis (LSA)
- Explicit Semantic Analysis (ESA)

You may also explore improvements to a search engine in terms of:

- Efficiency of retrieval
- Robustness to spelling errors
- Query auto-completion

You are expected to rigorously test your hypotheses using appropriate hypothesis testing methods. As an outcome of your work, you should be able to make a statement of the following form:

"An algorithm A_1 is better than A_2 with respect to the evaluation measure E

in task T on a specific domain D under certain assumptions A."

Note that, unlike earlier parts of the assignment, this component is open-ended and not restricted to the ideas mentioned above. For each method, your final report must include a critical analysis of results. Methods can be combined to develop improved systems; however, such hybrid approaches should be well-founded and not ad hoc (for example, simply combining multiple methods with tuned parameters without justification).

You may either build upon the template code provided earlier or develop your system from scratch, depending on your approach. While you are free to use any dataset for experimentation, the Cranfield dataset must be used for evaluation.

The project will be evaluated based on:

- Rigor in methodology
- Depth of understanding
- Quality of the report
- Performance in the viva if underperformed in the Written test of the Project

Report Structure

Your project report must include the following components:

1. An introduction to the problem setting
2. The limitations of the basic Vector Space Model (VSM) with appropriate examples from the dataset(s)
3. Your proposed approach(es) to address these issues
4. A description of the dataset(s) used for experimentation
5. The results obtained, along with a comparative study demonstrating improvements in the IR system, both qualitatively and quantitatively

The LaTeX template for the final report will be uploaded on Moodle. You are instructed to follow the template strictly.